



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Javascript server-side

Fabio Vitali

Corsi di laurea in Informatica
Alma Mater – Università di Bologna

Qui parleremo di...

Javascript server-side:

- Node + Express
- Mongo + Mongoose





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Node.js, npm ed Express.js

[XXX] across the stack

- Lo stack è l'insieme degli ambienti di cui un progettista web deve tener conto per realizzare un'applicazione, e dei linguaggi di programmazione da conoscere.
- Java across the stack:
 - *Google Web Toolkit*: ambiente integrato client-server, in cui la parte client viene compilata in Javascript da un sorgente Java
- Javascript across the stack:
 - *ASP* e poi *ASP.NET*: soluzione Microsoft per IIS. ASP era solo Javascript, mentre ASP.NET permette di scegliere tra molti linguaggi, incluso C#, VBA e Javascript.
 - *Google Apps Script*: soluzione Google per fornire comportamenti sofisticati ai siti realizzati con Google Sites.
 - *MongoDB, CouchDB*: database NoSQL forniscono meccanismi di interrogazione e gestione dei dati via Javascript
 - *NodeJS*: ambiente di esecuzione server-side di applicazioni sofisticate (anche indipendentemente da HTTP).



MEAN stack

- Un termine che adesso è MOLTO di moda è MEAN stack:
 - MongoDB: un database NoSQL che fornisce permanenza ai dati delle applicazioni
 - ExpressJs: un framework per applicazioni web
 - Angular: un framework per la creazione di template sofisticati di pagine HTML
 - NodeJs: una piattaforma software per applicazioni server-side
- Lo scopo evidente dello stack MEAN è di ripetere il successo e sostituirsi allo stack LAMP che ha costituito il set fondamentale di tecnologie per lo sviluppo di applicazioni web negli anni 2000.
- Stanno anche provando ad introdurre il termine MERN, con React al posto di Angular.



Node.js

- Node.js è un ambiente di esecuzione Javascript progettato per costruire applicazioni server-side efficienti
- Nato da un progetto individuale (di Ryan Dahl nel 2009) è sviluppato con un modello di governance aperto sotto la guida della Node.js Foundation
- E' alla base di un vasto ecosistema di moduli software e supportato da una vasta comunità open-source
- Questo ecosistema è gestito tramite npm (Node Package Manager)
- Si basa sul motore JavaScript V8, open-source, lo stesso motore usato da Google Chrome
- Javascript full stack



Single-thread non-blocking execution

Invece di dedicare un thread ad ogni connessione ricevuta dalla rete, l'applicazione nodeJS utilizza un solo thread per le richieste e tutte ricevono attenzione condivisa.

- In un'applicazione Web tradizionale un thread può avere circa 2Mb di memoria associati, quindi una macchina con 8GB ha un massimo teorico di 4000 richieste concorrenti.
- Mantenendo tutto in un unico stack, NodeJS può gestire decine o centinaia di migliaia di richieste concorrenti.

Poiché ogni richiesta può richiedere tempi variabili, è prassi inserire richieste onerose in funzioni asincrone, non-bloccanti, con una funzione di callback al completamento.

- Questo permette al thread principale (main loop) di rimanere molto snello e veloce.
- Per contro, è facilissimo inchiodare NodeJS facendo eseguire parti onerose del codice direttamente dentro al main loop.



NodeJS come motore HTTP

NodeJS può elegantemente sostituire PHP, Python, Perl e ogni altro approccio cgi-bin tradizionale. Di seguito un esempio di un'applicazione che restituisce un frammento HTML:

```
var http = require('http');

http.createServer(
  function (request, response) {
    response.writeHead(200, {'Content-Type': 'text/html'});
    response.end('<h1>Hello World</h1>');
  }
).listen(8000);
```



NodeJS con altri protocolli

NodeJS può porsi in ascolto su qualunque porta e implementare anche protocolli diversi da HTTP:

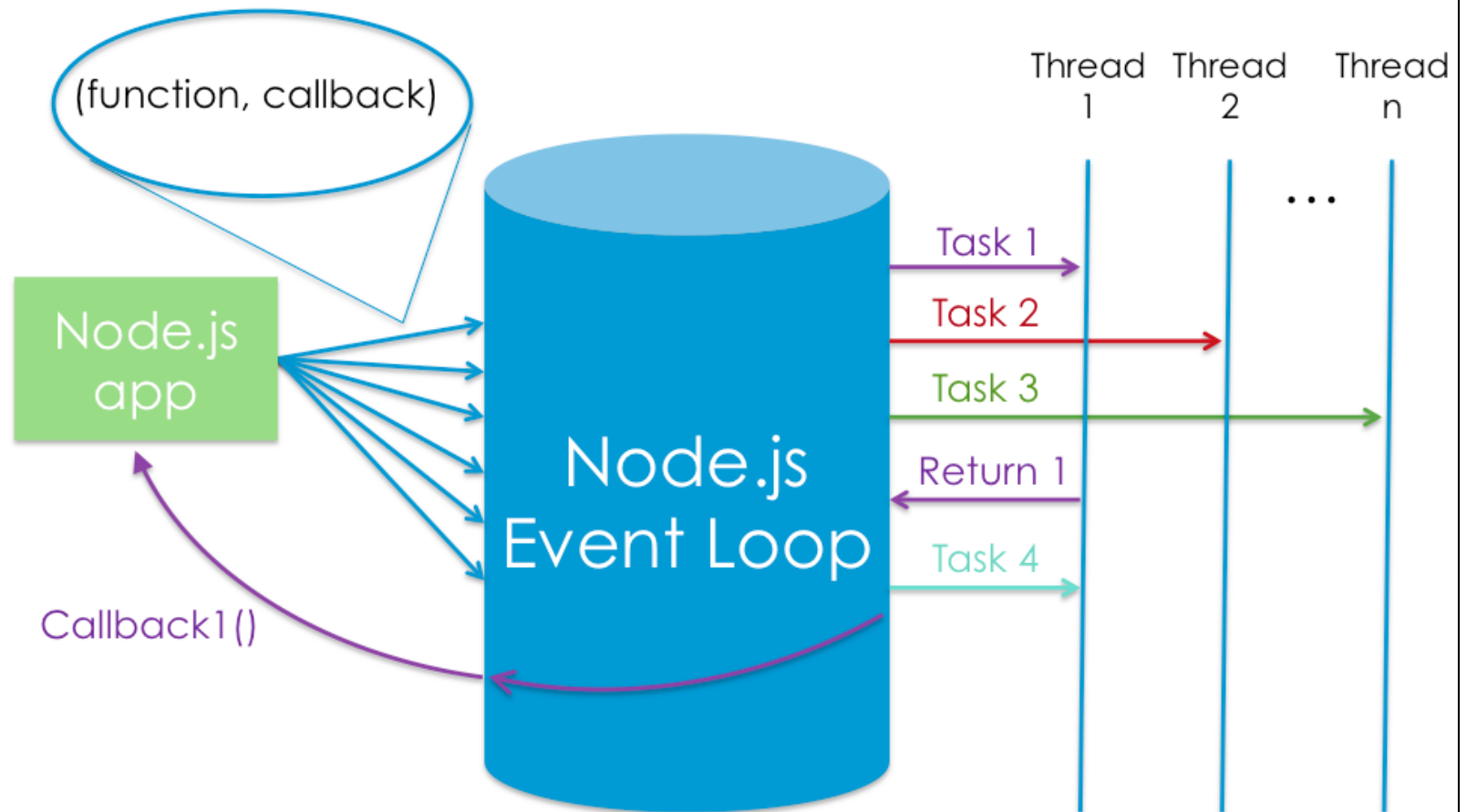
```
var net = require('net');
net.createServer(
  function (stream) {
    stream.write('hello\r\n');
    stream.on('end',
      function () {
        stream.end('goodbye\r\n');
      }
    );
    stream.pipe(stream);
  }
).listen(8000);
```



Event Loop

- 1** Node apps pass async tasks to the event loop, along with a callback

- 2** The event loop efficiently manages a thread pool and executes tasks efficiently...



- 3** ...and executes each callback as tasks complete

Event Loop (2)

- Event Loop non è una libreria esterna ma un elemento *core* del motore Node.js
- Node.js entra nell'Event Loop quando viene eseguito lo script ed esce quando non ci sono più funzioni di callback pendenti da invocare
- Internamente usa la libreria *libuv* for operazioni di I/O asincrone, che a sua volta mantiene un pool di thread
- La comunicazione avviene tramite **emissione di eventi**
- Gli oggetti che possono generare eventi (Event Emitter) espongono una funzione che permette di associare altre funzioni ad eventi che quell'oggetto genererà
`eventEmitter.on(eventName, listener)`
- Al verificarsi dell'evento queste funzioni saranno invocate in modo sincrono



Asincronia e funzioni callback

- Node.js è quasi esclusivamente basato su funzioni asincrone e callback.
- La convenzione di node.js suggerisce di creare funzioni che accettano una funzione callback asincrona come ultimo parametro
- La funzione callback per convenzione usa la sintassi error-first:

```
fs.readFile('test.txt', function(err, data) {  
    if(err) {  
        console.log('Error: ' + err.message);  
        return;  
    }  
    doSomething(data);  
});
```



La sintassi error-first

- Gestire gli errori nella programmazione asincrona può essere difficile, perché l'errore può avvenire in qualunque punto del tempo e del codice, e la sintassi error-first aiuta ad occuparsi innanzitutto delle situazioni problematiche
- Concentrarsi sulle situazioni d'errore nelle prime righe del codice aiuta nella programmazione paranoica, perché attira l'attenzione del programmatore sulle situazioni di errore invece che lasciare che si concentri solo sullo stato ideale del flusso del codice.
- **Attenzione:** è una questione puramente stilistica: la sintassi error-first non permette niente che non fosse già permessa senza, e non impedisce niente che non fosse già impossibile senza.



Moduli Node.JS

- I moduli in Node.js – o meglio in Javascript - permettono di includere altri file JS nelle applicazioni e di riutilizzare librerie esistenti
- Favoriscono l'organizzazione del codice in parti indipendenti e riutilizzabili
- L'enorme quantità di moduli disponibili gratuitamente ha contribuito al successo di Node.js
- Node.js ha un sistema di caricamento dei moduli semplice ma potente:
 - un modulo è un file Javascript
 - quando si include un modulo, questo viene cercato localmente o globalmente
- Per includere un modulo si usa la keyword **require (<modulo>)**



Caricare moduli (1)

- Esistono tre tipi di moduli:
 - **core**: built-in nel sistema, non è necessario installarli separatamente
 - **dipendenze locali**: installati per l'applicazione corrente nella directory `./node_modules/`
 - **dipendenze globali**: disponibili per tutte le applicazioni e installati nelle directory globali specificate nella variabile d'ambiente `NODE_PATH`
- Un modulo può essere caricato specificando il percorso o il nome:
 - `foo = require('./lib/bar.js');`
 - `foo = require('bar')`
- L'interprete cerca il modulo tra quelli *core*, poi tra le dipendenze locali e poi globali
- Le dipendenze locali e globali si installano via *npm* (vedi prossime slide)



Creare un modulo (sintassi AMD)

```
greetings = require("./greetings.js")  
greetings.hello()  
greetings.ciao()
```

```
hello = function() {  
    console.log("\n Hello! \n")  
}  
ciao = function() {  
    console.log("\n Ciao! \n")  
}  
module.exports.hello = hello;  
module.exports.ciao = ciao;
```

greetings.js



Creare un modulo (sintassi ES)

```
import greetings from "./greetings.js"
greetings.hello()
greetings.ciao()
```

```
hello = function() {
  return("\n Hello! \n")
}
ciao = function() {
  return("\n Ciao! \n")
}
greetings = {hello, ciao}
export default greetings;
```

greetings.js



npm

- I moduli di node.js vengono distribuiti ed installati con npm (*node package manager*)
- npm viene eseguito via command-line e interagisce con il registro npm.
 - meccanismo robusto per gestire dipendenze e versioni dei pacchetti (moduli)
 - semplice processo di pubblicazione di pacchetti e condividerli con altri utenti.
- In più: una piattaforma per repository pubblici e privati, servizi enterprise (integrati con firewall aziendali), integrazione con altre piattaforme, ecc.



Creare un pacchetto per *npm*

- Un pacchetto *npm* è un insieme di file Node.js, tra cui il file manifest **package.json** che specifica alcuni metadati del pacchetto, tra cui *nome*, *autore*, *dipendenze*, ecc.
- Il seguente comando eseguito nella directory che contiene gli script permette di creare il manifest attraverso un'interfaccia testuale interattiva:

– **npm init**

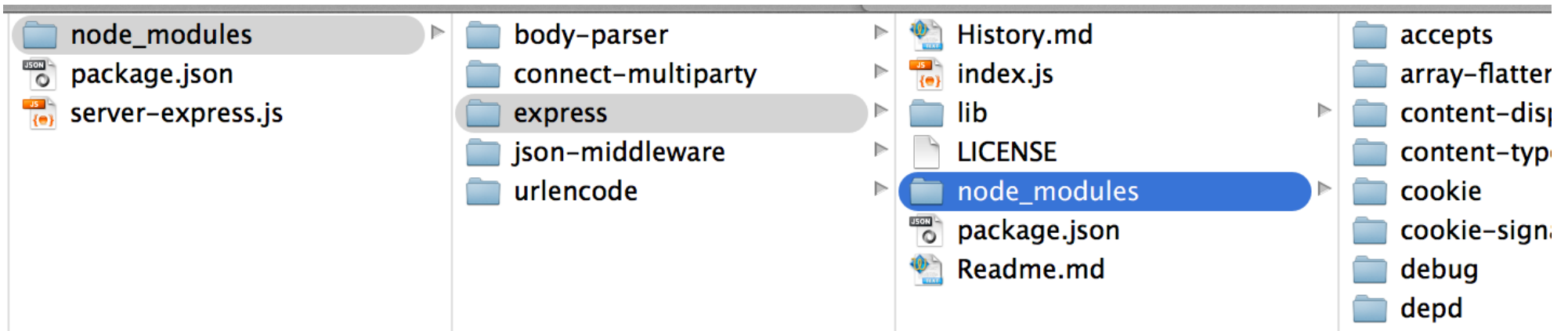
```
{
  "name": "My app",
  "version": "1.0.0",
  "description": "Nodeplayground",
  "main": "myapp",
  "author": "Fabio Vitali",
  "license": "MIT"
}
```

Installare un pacchetto

- Una volta creato un package si possono installare le dipendenze locali:
 - `npm install express`
- Il comando modifica il file *package.json*
- Di default i pacchetti sono installati nella directory locale:
 - `./node_modules/`
- Note:
 - nelle versioni precedenti alla 5.0 era necessario specificare il parametro `-save` per installare i pacchetti in questa directory, ora non è più necessario
 - l'opzione `-g` permette invece di installarli globalmente (ovviamente ha bisogno dei permessi)



package.json



```
"scripts": {  
  "test": "test-server-express.js"  
},  
"author": "Fabio Vitali",  
"license": "MIT",  
"dependencies": {  
  "body-parser": "^1.15.2",  
  "connect-multiparty": "^2.0.0",  
  "express": "^4.14.0",  
  "json-middleware": "^1.0.2",  
  "urlencode": "^1.1.0"  
}  
}
```

Alcuni comandi per gestire i pacchetti

- `npm list`
- `npm config list`
- `npm search express`
- `npm install express`
- `npm install express -global`
- `npm install express@1.0.2`
- `npm uninstall express@1.0.3`
- `npm update express`
- `npm init`
- ..



Altri runtime server-side

- Deno

- Creato nel 2018 da Ryan Dahl, l'autore Node.js. Runtime opinionato.
- Esegue TypeScript direttamente, senza compilazione preventiva.
- Usa solo ES Modules (`import`), non supporta `require`.
- Ha un modello rigido di sicurezza a permessi: nessun accesso a file, rete o variabili d'ambiente senza autorizzazione esplicita.
- Non usa `node_modules`: i moduli si importano tramite URL.
- Non dipende da `npm`: non c'è un repository anche se ne stanno creando uno alternativo (JavaScript Registry, JSR)

- Bun

- Prima versione del 2023, comprato il 2 dicembre 2025 da Anthropic, basato su JavaScriptCore (Safari) invece che su V8 di Chromium
- ultrarapido (70k HelloWorld requests al secondo, vs. 25k, Node e 40k, Deno).
- Unisce in un unico strumento runtime JavaScript, package manager, bundler, test runner
- Molto compatibile con Node.js e npm.





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Express.js

Applicazioni server-side

- La parte server-side di un'applicazione Web può essere scritta in diversi linguaggi di programmazione e diversi framework, che ne semplificano scrittura, manutenzione e deploy
- Ogni soluzione ha le sue specificità ma tutte le applicazioni server-side svolgono alcune operazioni:
 1. Lettura richiesta HTTP, parametri e query string
 2. Autenticazione (se richiesta)
 3. Persistenza dati
 4. Processing dati
 5. Generazione risposta HTTP
 - File statici: immagini, video, audio, fogli di stile, ecc. già disponibili sul server
 - HTML, XML: contenuti pronti per la visualizzazione
 - JSON, XML, CSV, ecc.: dati da processare e visualizzare client-side



Express.js

- Express è un modello di server web per node.js
 - Open-source, licenza MIT
 - molto usato e molto supportato dalla comunità
 - molto semplice e molto espandibile con plugin
- Express si dedica al routing, alla gestione delle sessioni, all'autenticazione degli utenti, e molti altri principi fondanti delle connessioni HTTP.
- Nato nel 2010, acquistato nel 2015 da IBM, poi regalato da IBM alla Node.js Foundation.



Basic server

```
const express = require("express")
const app = express()

docs_handler = function(request, response) {
    var docs = {server : "express"}
    response.json(docs);
}

app.get("/docs", docs_handler);

app.listen(8099, function() {
    console.log("\nExpress is running!!! \n")
})
```



Callback e oggetti res e req

```
processaRichiesta = function(request, response) {  
  
    // legge i dati dall'oggetto request  
    var name = request.params.name;  
  
    // produce il risultato  
    // ad esempio collegandosi a un database  
    ...  
    const doc = doSomething(name);  
  
    //scrive il risultato nell'oggetto response  
    //in formato JSON  
    response.json(doc);  
}
```



Routing

Express fornisce una semplice interfaccia per fare routing:

```
app.method(path, function(request, response) { ... })
```

dove

- `method` è uno dei metodi HTTP (`get`, `post`, `put`, `delete`, ecc.)
- `path` è il local path dell'URI richiesto al server
- `function(req, res)` è un handler da eseguire quando viene richiesto il path. I due parametri contengono gli object della richiesta HTTP (uri, intestazioni, parametri, dati, ecc.) e della risposta HTTP in via di restituzione (intestazioni, dati, ecc.)

Ogni route gestisce una o più handler anche sugli stessi path.

```
app.get('/', function (req, res) {  
  res.send('<p>Richiesta GET</p>');  
});  
app.post('/', function (req, res) {  
  res.send('<p>Richiesta POST</p>');  
});
```



Route paths

- I *route paths* possono essere stringhe o espressioni regolari
- Express.js controlla l'URL della richiesta e se individua un "match" invoca l'handler opportuno
- Esempi
 - `*ab*` : tutti i percorsi che contengono `ab`
 - `abc?de: abcde` oppure `abde`
 - `abc+: abc, abcc, abccc` così via (almeno una `c` finale)
 - `store$: path che termina con store`



Routing parametrico

E' possibile modularizzare il routing soprattutto per siti molto complessi o pieni di URI diversi. Tre casi:

- **URI regex.** Questa route accetterà URI come `abcd`, `abbc`, `abbbcd`, ecc.

```
app.get('/ab+cd', (req, res) => {  
  res.send('ab+cd')  
})
```

- **URI parametrici:** Questa route accetta URI di API REST tipiche:

```
app.get('/users/:userId/sales/:saleId', (req, res) => {  
  res.send(req.params)  
})
```



Routing parametrico

Handlers di secondo livello.

Usando un oggetto Router posso delegare parte della responsabilità dei route a moduli indipendenti.

```
// save as informatica.js
const express = require('express')
const router = express.Router()

router.get('/', (req, res) => {
  res.send('Home page dipartimento Informatica')
})
router.get('/about', (req, res) => {
  res.send('A proposito del dipartimento')
})
module.exports = router
```

questo modulo
risponde a URI del
tipo [/cs/](#) e [/cs/about](#)

```
const info = require('./informatica.js')

// ...
app.use('/cs/', info))
```


Esempio routing REST

```
app.get("/clients/", getListaClienti);  
  
app.post("/clients/", aggiungiCliente);  
  
app.put("/clients/:id", creaModificaCliente);  
  
app.get("/clients/:id", getClienti);  
  
app.delete("/clients/:id", cancellaCliente);
```

Nota: l'esempio mostra solo il routing, da completare con le callback



Oggetti Request e Response

- Le classi Request e Response rappresentano le richieste e le risposte HTTP ed espongono i metodi per accedere a tutte le loro informazioni
- Per le richieste:
 - body, query string, parametri, cookie, header, ecc.
- Per le risposte:
 - Metodi per spedire lo status code e la risposta in diversi formati
 - Metodi per aggiungere gli header



Accedere ai dati di un POST

- La libreria bodyparser (integrata con express) permette di accedere al corpo dei dati spediti dal POST nel body della richiesta.
- Questi middleware si occupano di recuperare e elaborare i dati sottomessi da un form via post e li convertono in oggetti accessibili dall'applicazione.
- I dati del POST si accedono come:
`<request>.body.<post_variable>`



Spedire una risposta

Method	Description
<code>res.download()</code>	Prompt a file to be downloaded.
<code>res.end()</code>	End the response process.
<code>res.json()</code>	Send a JSON response.
<code>res.jsonp()</code>	Send a JSON response with JSONP support.
<code>res.redirect()</code>	Redirect a request.
<code>res.render()</code>	Render a view template.
<code>res.send()</code>	Send a response of various types.
<code>res.sendFile()</code>	Send a file as an octet stream.
<code>res.sendStatus()</code>	Set the response status code and send its string representation as the response body.

Tabella da: <https://expressjs.com/en/guide/routing.html>



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Middleware in Express

- Express ha pochissime funzionalità proprie (routing), ma utilizza un grande numero di librerie middleware personalizzabili in uno stack di servizi progressivamente più complessi.
- Per aggiungere un middleware allo stack:
`app.use(<middleware>)`
- **Un'applicazione Express è allora essenzialmente una sequenza di chiamate a funzioni di middleware tra la richiesta e la risposta.**
- Il middleware può:
 - accedere agli oggetti di richiesta e risposta
 - cambiarli ed eseguire del codice di modifica
 - chiamare la prossima funzione del middleware (`next()`)
 - uscire dal ciclo e mandare la risposta.



File statici in Express.js

- Express.js può essere usato anche per restituire file statici memorizzati sul server
- Questa funzionalità si realizza aggiungendo un apposito middleware, che associa ad un path la directory in cui recuperare i file
- E' possibile specificare path e directory diverse

```
express = require("express")  
app = express()  
  
app.use("/images", express.static('images'));  
app.use(express.static('public'));
```



POST data

```
var express = require("express");

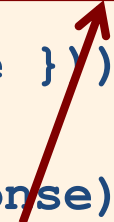
var app = express();

app.use(express.json());
app.use(express.urlencoded({ extended: true }));

app.post('/login', function (request, response) {
    console.log("Username:" + request.body.user);
    console.log("Password:" + request.body.pwd);

    // Continue and do authentication...
});
```

'user' / 'pwd':
nomi degli input usati nel form



Express middleware stack

- Express ha pochissime funzionalità proprie (routing), ma utilizza un grande numero di librerie middleware grandemente personalizzabili in uno stack di servizi progressivamente più complessi.
- Per aggiungere un middleware allo stack:
`app.use(<middleware>)`
- Un'applicazione Express è allora essenzialmente una sequenza di chiamate funzioni di middleware tra la richiesta e la risposta.
- Il middleware può
 - accedere agli oggetti di richiesta e risposta
 - cambiarli ed eseguire del codice di modifica
 - chiamare la prossima funzione del middleware (`next()`)
 - uscire dal ciclo e mandare la risposta.



Routing modulare

- La class `express.Router` permette di creare *route handlers modulari* e che possono essere combinati
- Anche in questo caso si aggiungere un middleware responsabile di gestire il routing per un insieme di path

```
var express = require('express')
var router = express.Router()

router.get('/', function (req, res) {
  res.send("TODO. Elenco utenti")}

router.get('/:name/', function (req, res) {
  res.send("TODO. Utente " + req.params.name)})

module.exports = router
```

```
usersRouter = require('./usersRouter.js');
app.use("/users/", usersRouter);
```



Express.js e autenticazione

- In Express.js l'autenticazione è realizzata con appositi middleware
- Come negli altri casi, devono essere installati, inclusi (con `import`) e aggiunti alla propria applicazione (con `use`)
- Uno dei più usati è Passport.js:
 - Flessibile e modulare
 - Supporta diverse strategie di autenticazione tra cui HTTP basic e digest, e JWT
- Per aggiungere il supporto a JWT invece si usa il pacchetto **express-jwt**:
 - <https://www.npmjs.com/package/express-jwt>



Express.js e CORS

- In Express.js si usa il middleware *cors* per aggiungere gli header **Origin** e **Access-Control-Allow-Origin** e supportare CORS
- Necessario includere il modulo e aggiungere il middleware
- Permette di abilitare le successive richieste Ajax su tutti i domini o su domini specifici e di specificare altre opzioni attraverso oggetti JS passati in input

```
cors = require('cors');  
app.use(cors())  
app.options('*', cors())
```

- Pacchetto npm: <https://www.npmjs.com/package/cors>





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Alcuni moduli NPM preinstallati

npm

- I moduli di node vengono distribuiti ed installati con npm (*node package manager*)
- npm viene eseguito via command-line e interagisce con il registro npm.
 - meccanismo robusto per gestire dipendenze e versioni dei pacchetti (moduli)
 - semplice processo di pubblicazione di pacchetti e condividerli con altri utenti.
- In più: una piattaforma per repository pubblici e privati, servizi enterprise (integrati con firewall aziendali), integrazione con altre piattaforme, ecc.



Usare npm

Un pacchetto npm è semplicemente un insieme di file, il più importante dei quali è *package.json*, che contiene metadati sul pacchetto (contenuto, dipendenze, ecc.)

Comandi utili di npm:

- npm init
 - Genera un file package.json globale nella directory in cui si trova. Tipicamente è la directory in cui eseguire node.
 - Viene anche creata la directory node_modules in cui verranno posizionati i package installati.
- npm install [package]
 - installa il pacchetto specificato, comprese tutte le dipendenze specificate, nella directory node_modules
- npm uninstall [package]
 - rimuove il pacchetto specificato, comprese tutte le dipendenze specificate e non usate da altri pacchetti installati
- npm update [package]
 - aggiorna esplicitamente il pacchetto specificato, comprese tutte le dipendenze specificate.



Express/cors (preinstallato)

Permette di definire server Express che realizzano completamente le funzionalità CORS (*Cross Origin Resource Sharing*).

- Per ragioni di sicurezza, un documento HTML può ricevere risorse (immagini, dati Ajax, script, ecc.) solo da un server posto nello stesso dominio del documento HTML di origine (*Same Origin Policy*).
- E' a discrezione del server permettere ad un'applicazione di un dominio diverso di richiedere le proprie risorse (*Cross Origin Resource Sharing*).
- Il browser, subito prima di eseguire un collegamento Ajax ad una risorsa di un dominio diverso, esegue un OPTION al server.
- Affinché sia poi possibile eseguire la richiesta, il server deve rispondere con alcune intestazioni HTTP specifiche, altrimenti il browser non effettuerà la richiesta vera e propria.
- Tipicamente: **Access-Control-Allow-Origin: ***

Il package express/cors inserisce automaticamente la risposta cors più appropriata e gestisce la creazione di servizi server-side aperti a tutti i richiedenti.



Handlebar.js (preinstallato)

- Handlebar.js è una semplice estensione di Mustache.js
- Un linguaggio di template logicless per tenere viste e codice separati
- Permette di accedere agli oggetti annidati dentro alle variabili da interpolare e di ottenere l'effetto di cicli e istruzioni condizionali usando bocchi contestuali.
- <https://handlebarsjs.com/>



Passport.js (non pre-installato)

- L'autenticazione può venire realizzata da package aggiuntivo di express.
 - Ce ne sono dozzine
 - Alcune autenticazioni base sono incluse direttamente in Express
 - Ad esempio *HTTP basic digest authentication*
- Passport.js è uno dei package più adottati
 - Flessibile e modulare
 - Supporta numerosissimi modelli di autenticazione: HTTP digest, Facebook, Google, Twitter, LinkedIn, ecc., chiamati *strategie*.
 - Permette anche di aggiungere nuove strategie ad-hoc.



dotenv (non preinstallato)

- Dal punto di vista della sicurezza, il client va sempre considerato come potenzialmente compromesso. Tutte le informazioni delicati vanno gestite sul server.
- Il modulo dotenv gestisce informazioni in un file sul server. Il file, `.env`, serve per separare il codice dai dati sensibili.
 - password
 - chiavi API
 - URI completi dei database
 - token

Contenuto del file `.env`:

```
DB=mongodb://user:pwd@host:27017/myDB
API_KEY=abc123
PORT=3000
```

```
require("dotenv").config();
```

```
const uri = process.env.DB;
const port = process.env.PORT;
```



Push, pull e poll

- Pull
 - il modello originario di HTTP: il client chiede → il server risponde → la comunicazione termina.
 - Il server non può parlare se non viene interrogato.
 - È il modello naturale di pagine web classiche, API REST, fetch/axios.
 - Il pull funziona bene quando i dati cambiano raramente e non serve sapere subito che qualcosa è cambiato.
 - Diventa insufficiente quando servono aggiornamenti immediati.
- Poll
 - un trucco per simulare il push usando un protocollo pull.
 - Il client ripete una richiesta ogni N millisecondi:
`setInterval(() => fetch("/status"), 2000)`
 - Il server è passivo, ma viene interrogato anche se non è cambiato nulla.
 - Il poll è semplice, funziona ovunque, non richiede protocolli speciali.
 - Però c'è spreco di banda, spreco di CPU, ritardo tra un evento e il successivo controllo.



Push, pull e poll

- Push
 - Il client apre una connessione e si pone in attesa.
 - Il server invia dati quando succede qualcosa, senza essere interrogato.
 - Il client si attiva solo quando arriva qualcosa dal server.
 - Chat
 - Notifiche
 - Aggiornamenti in tempo reale
 - Tracciamento eventi, sensori, streaming di stato
- Server-Sent Events (SSE) e WebSocket sono tecnologie che permettono al Web di superare modello a pull di HTTP e implementare un modello push.



Push, pull e poll

Server-Sent Events

- API nel browser
 - Solo server → client.
 - Basata su HTTP standard.
 - Il client apre una connessione e resta in ascolto.
 - Il server invia messaggi quando ha nuovi dati.
- Pro
 - Molto semplice.
 - Compatibile con proxy e firewall.
 - Ideale per notifiche, log, avanzamento processi.
- Contro
 - Non è bidirezionale.
 - Non adatto a chat o giochi in tempo reale.

Codice del client

```
const evt = new EventSource("/events");
evt.onmessage = (e) => {
  showData(e.data);
};
```

Codice del server

```
app.get("/events", (req, res) => {
  res.setHeader("Content-Type", "text/event-stream");
  res.setHeader("Cache-Control", "no-cache");
  res.setHeader("Connection", "keep-alive");

  const timer = setInterval(() => {
    res.write(getData());
  }, 1000);

  req.on("close", () => { clearInterval(timer); });
});
```



Push, pull e poll

WebSocket

- API del browser + libreria npm
 - Comunicazione bidirezionale.
 - Connessione persistente con protocollo proprio (upgrade da HTTP).
 - Ad esempio ws, socket.io, uwebSockets.js
 - Client e server possono inviare messaggi in qualsiasi momento.
- Pro
 - Comunicazione in tempo reale bidirezionale.
 - Latenza molto bassa.
 - Ideale per chat, giochi, sistemi collaborativi, streaming interattivo.
- Contro
 - Più complesso da gestire.
 - Meno compatibile con proxy e firewall restrittivi.

Codice del client

```
const socket = new WebSocket("ws://[hostIP]:3000");
const msg = document.getElementById("msg")

socket.onopen = () => { console.log("Connesso"); };
socket.onmessage = (e) => { msg.value = e.data; };

msg.addEventListener("input", (e) => {
  const text = e.target.value;
  socket.send(text);
});
```

Codice del server

```
const http = require("http");
const WebSocket = require("ws");
const server = http.createServer();
const wss = new WebSocket.Server({ server });

wss.on("connection", (ws) => {
  console.log("Client connesso");
  ws.on("message", (msg) => {
    let newData = doSomething(msg)
    ws.send(newData);
  });
});
server.listen(3000);
```

nodemon

- Una semplicissima, utilissima utility per node.
- Node.js fa prefetch e cache degli script da eseguire. Quindi ogni modifica sugli script non viene percepita ed usata da node.js fino al riavvio, che va fatto a mano ogni volta.
- In fase di debugging, questo significa che node va riavviato a mano ogni pochi minuti, che può essere sgradevole e pesante.
- Nodemon tiene sotto controllo tutti i file di una directory, e ogni volta che percepisce il salvataggio di una modifica riavvia node.js automaticamente.
- Attenzione!
 - Voi scrivete il file su una delle macchine del laboratorio;
 - Il file system di gocker viene avvisato della modifica dopo 2-3 secondi;
 - Nodemon si accorge della modifica dopo 1 secondo e riavvia node.js;
 - Node.js richiede 3-4 secondi per riavviarsi.
- Nel frattempo il vostro sito darà errore 500 Internal Server Error
- Ci vogliono mai meno di 5 secondi, e a volte anche 10 secondi per rivedere il vostro sito online!!!





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Mongo e Mongoose

Storage dei dati

- In generale, un'applicazione web, anche data-oriented, non ha bisogno di un DB né SQL né di altro tipo, poiché tutti i linguaggi server-side (da Perl a PHP a node.js) forniscono metodi per utilizzare nativamente il file system e la applicazione può scrivere i suoi dati persistenti su un file qualunque.
- Si usa un DB:
 - quando il file diventa troppo grande per stare tutto in memoria,
 - quando ho bisogno di metodi veloci per cercare un sottoinsieme di dati all'interno del file,
 - quando ho bisogno di gestire accessi concorrenti in scrittura.
- Usare un DB per leggere, ma non modificare, un file intero di dimensioni non enormi è come comprare una Ferrari per andare a far la spesa. Funziona, ma è complicato, ed è uno spreco di tempo e risorse.



SQL vs noSQL

Database SQL (relazionali)

- Dati organizzati in tabelle con schema rigido.
- Struttura definita prima di inserire i dati.
- Relazioni tra tabelle tramite chiavi.
- Usano tutti un linguaggio standard di query: SQL.
- Esempi: MySQL, PostgreSQL, Oracle.
- Forte coerenza dei dati.
- Ottimi per dati molto strutturati e con poche eccezioni

Database NoSQL (non relazionali)

- Dati senza schema fisso o con schema flessibile.
- Strutture a documenti, oggetti, coppie chiave-valore, grafi.
- Non esiste un linguaggio di query unico SQL standard unico.
- Grande flessibilità e scalabilità orizzontale su più server.
- Ottimi per dati eterogenei, dinamici, senza struttura.



MongoDB - Introduzione

- MongoDB è un database noSQL orientato ai documenti.
- Esso memorizza i documenti in JSON, formato basato su JavaScript e più semplice di XML, ma comunque dotato di una buona espressività.
- I documenti sono raggruppati in collezioni che possono essere anche eterogenee. Ciò significa che non c'è uno schema fisso per i documenti. Tra le collezioni non ci sono relazioni o legami garantiti da MongoDB.
- MongoDB si adatta a molti contesti, in generale quando si manipolano grandi quantità di dati eterogenei e senza uno schema.
- MongoDB fornisce un driver per JavaScript **lato server** ossia per **Node.js**. Sulle macchine di cs è preinstallato, sulle vostre dovrete installarlo di persona.



Concetti

- **Le collezioni** in Mongo sono la struttura fondamentale di Mongo, un po' tabella relazionale, un po' directory di file.
- **I documenti** sono strutture di contenimento di dati omogenei, come se fossero righe di una tabella.
- **I campi**, o proprietà o attributi, sono le celle della riga. Contengono valori semplici di riferimento al documento.
- **Schemi**: Mongo non ha schemi, ma attraverso Mongoose è possibile definire uno schema e forzare omogeneità tra i vari documenti di una collezione
- **Schema type**: Oltre a definire la sequenza e i nomi dei vari campi, Mongoose permette di definire anche i tipi di dati di ciascun campo. (String, Number, Boolean, ecc).
- **I Modelli** sono costruttori di alto livello che generano istanze di documenti a partire da uno schema. Garantiscono l'omogeneità strutturale tra documenti.



MongoDB - CRUD

- Un database è un insieme di collezioni che corrisponde ad un insieme di file nel disco fisso.
- Un'istanza di MongoDB può gestire più database. Di seguito mostreremo brevemente come effettuare le operazioni CRUD (Create, Read, Update, Delete).
- Su MongoDB, non esiste alcun comando esplicito per creare un database.
- Anche le collezioni vengono create implicitamente al primo utilizzo.



MongoDB - CRUD

- Per inserire dati:

```
const client = new MongoClient(uri);
try {
  let dbname = "mySite";
  let coll = "myCollection";
  let data = {
    name: "Fabio Vitali",
    age: 28,
    url: "http://www.fabioitali.it",
    tags: ["Development", "Design"]
  };

  let result = await client
    .db(dbname)
    .collection(coll)
    .insertOne(data);
} catch (e) {
  alert("something went wrong");
}
```

- La risposta corretta del server sarà:

```
{
  "acknowledged": true,
  "insertedCount": 1,
  "insertedIds": {
    "0": "62694126e084590335f12f44"
  }
}
```

```
const client = new MongoClient(uri);
try {
  let dbname = "mySite" ;
  let coll = "myCollection" ;
  let data = [{
    name: "Fabio Vitali",
    age: 28,
    url: "http://www.fabioitali.it",
    tags: ["Development", "Design"]
  }, {
    name: "Angelo Di Iorio",
    age: 26,
    url: "http://www.angelodiorio.it",
    tags: ["Programming", "Systems"]
  } ] ;

  let result = await client.db(dbname)
    .collection(coll)
    .insertMany(data);
} catch (e) {
  alert("something went wrong")
}
```



MongoDB - CRUD

Per interrogare tutti i documenti presenti nella collezione:

```
client.db(dbname).collection(coll).find(query)
```

dove `query` è un oggetto complesso composto da valori, nomi di campo, e operatori (introdotti da `$`) e il nesting si ottiene attraverso contenimento.

SQL query	Mongo query
<pre>SELECT * FROM dbname WHERE name = "Fabio Vitali"</pre>	<pre>client.db(dbname).collection(coll).find({ name: "Fabio Vitali" })</pre>
<pre>SELECT * FROM dbname WHERE age > 18</pre>	<pre>client.db(dbname).collection(coll).find({ age: { \$gt: 18 } })</pre>
<pre>SELECT * FROM dbname WHERE name = "Fabio Vitali" age > 18</pre>	<pre>client.db(dbname).collection(coll).find({ name: "Fabio Vitali", age: { \$gt: 18 } })</pre>
<pre>SELECT * FROM dbname WHERE name LIKE "fabio%" OR age > 18</pre>	<pre>client.db(dbname).collection(coll).find({ \$or: { name: /^fabio/, age: { \$gt: 18 } } })</pre>

MongoDB - CRUD

L'aggiornamento dei documenti avviene tramite i metodi **updateOne**, **updateMany**(*filter, update, options*), e **replaceOne**:

Mongo update
<pre>client.db(dbname).collection(coll).updateOne({ name: "Fabio Vitali" }, { \$set: { age: 35 } })</pre>
<pre>client.db(dbname).collection(coll).updateMany({ age: { \$lt: 40 } }, { \$set: { age: 40, status: "presumed" } } })</pre>

MongoDB - CRUD

L'eliminazione di un documento dalla collezione viene effettuata per mezzo dei metodi **deleteOne** o **deleteMany**.

Mongo delete
<pre>client.db(dbname).collection(coll).deleteOne({ name: "Fabio Vitali" })</pre>
<pre>client.db(dbname).collection(coll).deleteMany({ age: { \$lt: 40 } } })</pre>



MongoDB - Server

Ad esempio, per accedere ad una collection su un database di MongoDB la sintassi è la seguente:

```
const express = require("express");
const { MongoClient } = require("mongodb");

const app = express();
const uri = process.env.DB_URI || "mongodb://user:pwd@host:27017";
const port = process.env.PORT || 3000;

app.use(express.json());

app.get("/api/users/search", search)
app.post("/api/users/promote", promote) ;

app.listen(port, () => {
  console.log("Server in ascolto su http://localhost:" + port);
});
```

MongoDB - Server

```
async function search(req, res) {
  const client = await MongoClient.connect(uri)
  try {
    const users = client.db("mySite").collection("users");

    const body = req.body || {};
    const query = { isActive: true };

    if (body.city) query.city = body.city;
    if (body.minScore) query.score = { $gte: body.minScore };
    if (body.interest) query.interests = { $in: [ body.interest ] };

    const results = await users.find(query).sort({ score: -1 }).toArray();
    res.json(results);
  } catch (e) {
    console.error(e);
    res.status(500).json({ error: "Internal server error" });
  }
  await client.close()
}
```

MongoDB - Server

```
async function promote(req, res) {
  const client = await MongoClient.connect(uri)
  try {
    const users = client.db("mySite").collection("users");
    const body = req.body || {};
    const query = { isActive: true };
    if (body.city) query.city = body.city;
    if (body.minScore) query.score = { $gte: body.minScore };
    const update = {
      $addToSet: { tags: body.tagToAdd || "promoted" }
    };
    const result = await users.updateMany(query, update);
    res.json({
      matched: result.matchedCount,
      modified: result.modifiedCount
    });
  } catch (e) {
    console.error(e);
    res.status(500).json({ error: "Internal server error" });
  }
  await client.close()
}
```

Mongoose

- Mongoose permette di imporre schemi sui documenti di una collezione Mongo, e creare omogeneità dove opportuno.

```
// save as people.js
let mongoose = require('mongoose');
let people = new mongoose.Schema({
  name: String,
  surname: String,
  email: String,
  age: Number
});

module.exports = mongoose.model('Person', people);
```

```
fv.save()
  .then((doc) => {
    console.log(doc);
  })
  .catch((err) => {
    console.error(err);
  });
```

```
let Person = require('./people.js');

let fv = new Person({
  name: "Fabio",
  surname: "Vitali",
  email: "fabio.vitali@unibo.it",
  age: 28
});
```



MongoDB e il progetto

- Bisogna lanciare MongoDB su un docker separato.
- Usate gocker per lanciare MongoDB.

`create mongoDB site2223XX`

- Il server risponde:

`Mongodb username: site2223XX - Mongodb password: YYYYYYY`

`You can connect your mongodb from your site web using hostname
mongo_site2223XX`

- Questi tre dati vanno conservati ed utilizzati dallo script su node.js per collegarsi con successo.
- Mongo non è accessibile all'esterno, ma solo alla vostra installazione gocker.



Dove mettere cosa?

Cosa va client-side?

Cosa va server-side?

Cosa si può scegliere?

- Sempre client-side
 - Manipolazione del DOM, UI, animazioni. Logica puramente di presentazione (mostra/nascondi, paginazione)
 - Gestione eventi utente (click, tasti, drag&drop).
 - Validazione “di cortesia” dei form (feedback immediato).
 - Logica utente: geolocalizzazione, notifiche, clipboard, media, ecc.
 - Accesso ad API del browser: window, document, ecc.
 - Storage locale non sensibile: localStorage, ecc.



Dove mettere cosa?

Cosa va client-side?

Cosa va server-side?

Cosa si può scegliere?

- Sempre server-side
 - Permanenza dei dati e storage: database e filesystem del server.
 - Autenticazione utente e autorizzazioni.
 - Gestione di credenziali e chiavi segrete: API key, token, password, etc.
 - Business rules che non devono essere visibili/manipolabili dal client.
 - Business rules che non debbono essere modificate dal client (il client è compromesso per definizione)
 - Log applicativi affidabili e audit trail.



Dove mettere cosa?

Cosa va client-side?

Cosa va server-side?

Cosa si può scegliere?

- Si può scegliere se client-side o server-side
 - Rendering HTML: tutto client-side (SPA), tutto server-side (SSR), ibrido.
 - Routing applicativo: router lato client, routing REST lato server, combinazioni.
 - Validazione dati (di solito: duplicata client+server).
 - Formattazione e trasformazione dati (sorting, filtri, ricerche non critiche).
 - Caching di dati non sensibili.
 - Parte della logica di state management



Conclusioni

- Client-side c'è una grossa spinta a usare le tecnologie più recenti e innovative, in quanto immediatamente visibili all'utente e di impatto.
- Server-side è tutto più sfumato: i progetti sono di più lunga durata, le esigenze di integrità strutturale e affidabilità più pressanti, i team più prudenti.
- La discussione su SQL vs. noSQL è ancora viva, e la sfiducia verso l'affidabilità e le prestazioni dei noSQL esiste ancora.
- Analogamente la scelta di JavaScript / TypeScript client-side è obbligata, mentre server-side linguaggi come Php, Python, Java ecc. sono ancora vivi e presenti.
- La varietà di tecnologie con cui avrete a che fare server-side è molto più ampia di quelle client-side.





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Fabio Vitali

Dipartimento di Informatica – Scienze e Ingegneria
Alma mater – Università di Bologna

Fabio.vitali@unibo.it

www.unibo.it